Systematic Evaluation of Control Mechanisms for Factor-Based Portfolio Management

Ioan Hedea

Preprint prepared in a NeurIPS-style conference template.

Abstract

Portfolio management systems often separate alpha generation from risk control, but the question of which control mechanism works best remains underexplored. This paper systematically compares 19 control approaches over a finance-first alpha engine built from factor, GARCH, and HMM sleeves, using frozen-bundle evaluation across 77 traded tickers and 13 years of daily data from April 2013 through March 2026. The compared methods include no control beyond the alpha engine, simple rules, contextual bandits, supervised control, CVaR-aware robust optimization, convexity-aware protection modes, a learned council meta-controller, an MLP-gated meta-controller, a CMDP-Lagrangian controller, and Q-learning RL. Results show that the strongest overall point-estimate trade-off comes from adding convexity to the robust optimizer: `D_plus_convexity` reaches mean Sharpe 0.85 and Calmar 0.94 versus 0.81 and 0.68 for plain CVaR-robust, 0.65 for the MLP-gated meta-controller, 0.64 for the logistic council, 0.42 for LinUCB, 0.41 for regime-aware rules, and 0.23 for Q-learning RL. The best single run, `D_plus_convexity_tf0.50`, reaches 21.62% annualized return with 1.03 Sharpe. Reference-split block-bootstrap tests already show that `D_plus_convexity` improves on SPY, LinUCB, regime rules, and the MLP meta-controller on Sharpe, while its edge over plain CVaR-robust is not yet decisive on that metric. No single method dominates every metric: volatility targeting and ensemble-min have the shallowest drawdowns, regime-aware rules remain the strongest low-complexity baseline, and learned meta-control is promising but still below the convexity-enhanced robust stack. The paper's contribution is therefore a risk-shaping framework for portfolio control rather than benchmark rhetoric: in this frozen-bundle evaluation, downside-aware outcome selection plus convex payoff transformation dominates the learned alternatives, while learning is most useful as meta-control rather than as the primary decision layer.

Keywords: portfolio management, factor investing, control mechanisms, robust optimization, convex risk measures, algorithm selection, contextual bandits, reinforcement learning, backtesting

1 Introduction

Many quantitative trading systems are modular long before they are learned. Cross-sectional factors, volatility estimates, regime filters, and constrained allocators already encode a large amount of domain structure before any control policy is added. The practical question, then, is not only how to forecast returns, but how much control complexity is justified once a finance-first alpha engine is already working. Fixed allocation, simple rules, contextual bandits, supervised controllers, robust optimization, and RL are all plausible answers to that question.

The present manuscript reflects a deliberate architecture revision. Earlier project versions were framed around whether RL worked as a controller. The March 29, 2026 frozen-bundle evaluation broadened that question into a direct comparison of control mechanisms. The resulting architecture keeps the proven alpha core, factor anchoring, and constrained allocator, but evaluates a wider set of downstream controllers under one shared data, timing, and cost protocol.

The revised implementation contributes five practical elements to the study.

1. A modular code architecture that cleanly separates data, alpha, control, backtesting, and visualization concerns.
2. A factor-anchored alpha layer limited to components that survived pruning: cross-sectional factors, GARCH volatility forecasts, and a two-state HMM regime sleeve.
3. A constrained optimization layer between alpha and control that turns signals into a capped, turnover-aware, and group-aware target book.
4. A control-method comparison spanning fixed exposure, rule-based control, contextual bandits, supervised control, CVaR-aware robust optimization, convexity-aware protection, council-style meta-control, MLP-gated meta-control, CMDP-style constrained learning, and RL comparators.
5. A frozen-bundle research workflow with ablations, control-comparison tables, rolling windows, significance tests, reward sweeps, and regime-conditioned diagnostics.

1.1 Research Questions

The current paper is organized around three revised research questions.

1. **RQ1: Control-method comparison.** Given a strong finance-first alpha engine, which controller adds the most value?
2. **RQ2: Competitive simple control.** Can simple rules provide competitive downside control without excessive Sharpe degradation?
3. **RQ3: Learning versus optimization.** Do learning-based methods outperform optimization-based control on top of the same alpha engine?

From a research perspective, this framing matters because it narrows the gap between a teaching prototype and a system that can be discussed in the language of quantitative portfolio construction. The objective is not to claim that the pipeline is production-ready. Rather, it is to assemble a well-motivated stack in which each modeling choice has a clear financial role and where performance can be decomposed into alpha generation, portfolio construction, and control.

The central defended statement is therefore not that one more controller happened to win. The paper's main contribution is a risk-shaping framework for portfolio control under shared alpha, timing, and cost assumptions. Within that framework, the finished frozen-bundle shows that the winning design combines two distinct operations: tail-aware selection of portfolio outcomes through CVaR and tail-aware transformation of realized payoffs through convex protection. The learning results then sharpen the boundary of the design space rather than overturn it: learning helps most as meta-control over already-credible structured experts, not as the primary decision layer.

2 Related Literature and Design Motivation

The report is not meant to claim novelty in any individual modeling choice. Instead, it builds a coherent pipeline from well-established ingredients.

Factor investing. The cross-sectional component is aligned with the idea that common risk factors help explain returns [9]. Momentum is motivated by the medium-horizon continuation evidence of Jegadeesh and Titman [14], while quality and low-volatility proxies are included to stabilize the cross-section.

Volatility and regimes. Conditional heteroskedasticity motivates the GARCH sleeve [3, 8], while the market-regime component uses a two-state hidden Markov model in the spirit of Hamilton [12].

Rule-based, bandit, and optimization-based control. A key part of the revised paper is that RL is no longer the only sophisticated controller under study. Volatility targeting, drawdown-based deleveraging, regime rules, contextual bandits, supervised controllers, and CVaR-aware robust allocation are all treated as serious alternatives because these are the kinds of methods one would compare before concluding that sequential RL is necessary.

Reinforcement learning. RL is used as a controller rather than a raw alpha generator. This follows the practical observation that in finance, RL is often most useful for allocation, inventory, and execution decisions rather than for replacing domain-specific signal construction [1, 25]. This role assignment is also motivated by the low signal-to-noise ratio and limited effective sample size typical of financial data, which make end-to-end policy learning especially brittle.

2.1 Recent Work on RL for Portfolio Management

Recent work on RL in portfolio management and trading can be organized into four overlapping strands.

End-to-end policy learning. Early and influential work uses RL to learn trading or allocation policies directly from price histories and realized rewards [5, 6, 15, 18, 20, 26]. The appeal of this approach is conceptual elegance: the same learner is asked to extract signal, size positions, and adapt sequentially. The difficulty is that financial returns are noisy, nonstationary, and data-poor relative to many other machine-learning domains. As a result, end-to-end RL systems can be sample-inefficient, regime-sensitive, and hard to interpret when they fail [11, 13, 16].

Neural return forecasting with a separate optimizer. A second line of work uses deep or other machine-learning models to estimate returns, factors, or pricing kernels, then hands those forecasts to a conventional optimizer [4, 10, 11]. This preserves more structure in the allocation step than end-to-end RL, and often performs well empirically, but it still concentrates modeling risk in a data-hungry prediction layer.

RL for execution and risk control. A third strand applies RL to narrower sequential tasks such as execution and dynamic risk management rather than to raw alpha discovery [2, 7, 17, 19]. This decomposition is closest to the present paper. The core idea is that RL may be most useful where the problem is inherently sequential and control-like, while alpha generation remains anchored to domain models or explicit financial structure.

Benchmarking and reproducibility. More recent platform and survey work has emphasized that RL in finance still suffers from weak evaluation discipline, narrow datasets, and inconsistent reporting [13, 24]. That critique directly motivates the frozen-bundle methodology used here.

Bandits and robust portfolio optimization. The revised study also touches a literature that sits between static rules and fully sequential RL. Contextual bandits offer a lighter exploration–exploitation framework for discrete allocation decisions, while CVaR-aware robust optimization provides a formal risk-sensitive alternative to learning when downside control is the primary objective. This broader framing is important because the central question is no longer whether RL wins, but which controller complexity is empirically justified.

Table 1 positions the present approach relative to these broader families. The contribution is not a new RL algorithm. It is a methodological proposal: if strong finance-first alpha already exists, the main research question should be a disciplined comparison of control mechanisms, with RL judged against simple rules and end-to-end learned alternatives rather than against passive exposure alone.

Table 1: Taxonomy of portfolio-management architectures.

Approach	Alpha source	Control logic	Interp.	Representative references
Classical factors	Hand-crafted financial signals	Fixed allocator or simple overlay rules	High	Fama–French style factor investing [9]
Rule-based overlays	Finance-first alpha models	Vol-targeting, deleveraging, and exposure rules	High	Simple portfolio-risk overlays in practice [1]
End-to-end RL	Learned from market histories	RL policy directly selects positions or weights	Low to medium	Direct trading and portfolio RL [5, 15, 18, 26]
Neural forecasting + optimizer	Neural or ML return forecasts	Separate optimizer translates forecasts into holdings	Medium	Asset-pricing and return-forecast models [4, 10, 11]
This paper	**Finance-first alpha models**	**Comparison of rule-based, bandit, supervised, robust, and RL control**	**High**	**Frozen-bundle control-method evaluation**

2.2 Contribution Type

The contribution is not novelty in any single alpha ingredient. The paper does not claim a new factor model, a new GARCH variant, or a new RL algorithm. Its contribution lies instead in three places. First, it proposes a clear role assignment between alpha and control: financially grounded models produce the alpha book, while controllers decide how aggressively to express it. Second, it implements that role assignment in a modular architecture with an explicit constrained allocator between alpha and control. Third, it pairs the architecture with a frozen-bundle evaluation workflow designed to test that claim via ablations, timing-discipline checks, robustness slices, significance tests, and regime-conditioned behavior summaries. The contribution is therefore chiefly methodological: the paper proposes and tests a disciplined way to evaluate portfolio control mechanisms on top of a finance-first alpha engine.

3 System Architecture

The pipeline is organized as a linear sequence of stages, illustrated in Figure 1. Upstream stages produce signals and risk features, the allocator turns those signals into a target book, and the control layer decides how much of that book to express before the backtest engine records realized outcomes.

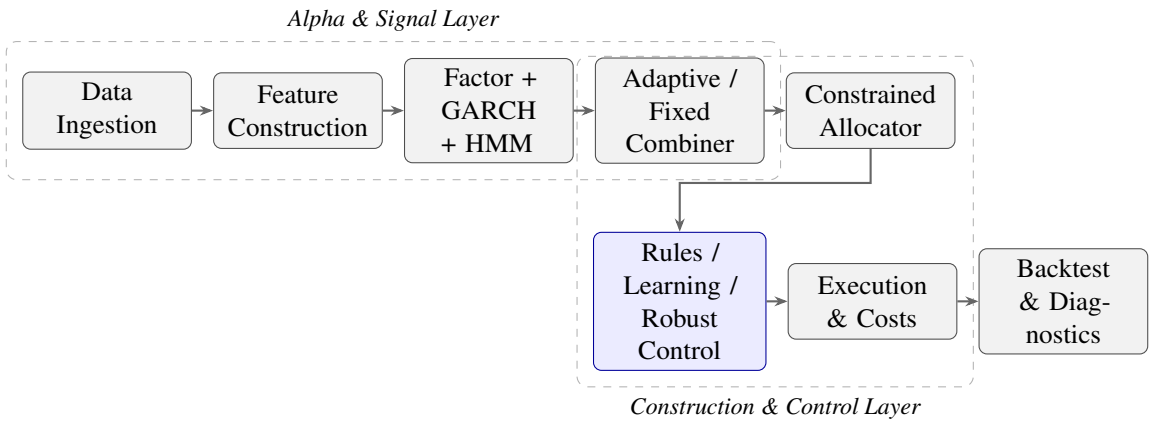

Figure 1: High-level architecture of the revised quantitative trading pipeline. The alpha layer is intentionally finance-first, the allocator builds a capped and turnover-aware target book, and the control comparison focuses on how much of that book to express rather than on a second stock-selection problem. The compared control families span simple rules, contextual bandits, supervised control, robust optimization, and RL.

3.1 Software Structure

Table 2 summarizes the package decomposition that supports the frozen-bundle workflow. The separation is intentional: alpha construction, control comparison, backtesting, and plotting are kept in distinct modules so that the reported artifact bundle can be regenerated and audited component by component.

Table 2: Module-level decomposition of the `quant_stack` package.

Module	Responsibility
`config.py`	Universe definition, global constants, and external series identifiers
`data.py`	Market price/volume ingestion, macro series (FRED / BLS fallback), and SEC company facts
`alpha.py`	Cross-sectional factors, GARCH volatility, HMM regime, and adaptive or fixed signal combination
`rl.py`	Portfolio controller, state construction, and end-to-end PPO benchmark interfaces
`pipeline.py`	Walk-forward backtest orchestration and online RL update loop
`evaluation.py`	Control-method comparison, ablation, rolling-window robustness, significance tests, and artifact export
`plots.py`	Equity-curve, drawdown, controller-behavior, robustness, and reward-ablation figures
`main.py`	CLI entry point and compatibility wrapper

4 Data and Feature Construction

4.1 Universe

The revised frozen-bundle uses the expanded universe rather than the smaller development basket. It contains 77 traded tickers spread across technology, financials, healthcare, consumer staples, energy, industrials,

materials, real estate, utilities, communications, consumer discretionary, and diversifier exposures such as GLD, TLT, and VNQ. SPY serves as the passive equity benchmark throughout.

4.2 Data Sources

Daily prices and volumes are sourced via `yfinance`. When a `FRED_API_KEY` is available, macro data are drawn directly from FRED; in its absence, the system falls back to BLS and U.S. Treasury Fiscal Data. SEC company facts are incorporated when `SEC_USER_AGENT` is set, enabling a light fundamentals layer without requiring a proprietary data subscription.

4.3 Macro Regime Inputs

When available, the macro sleeve includes the 10-year yield, 2-year yield, federal funds rate, unemployment rate, and term spread. The updated research design also incorporates stress-sensitive FRED series including the VIX, the ICE BofA high-yield option-adjusted spread, and a dollar-index proxy. These variables do not form a standalone alpha book. Instead, they help create a macro regime belief that is blended with the HMM regime estimate and provide additional context for risk-off states.

4.4 Information-Timing Contract

One of the most important research upgrades in the current version is that the pipeline makes its feature-availability assumptions explicit. Macro inputs are shifted forward by a configurable number of trading days before they are allowed into the backtest. This is still weaker than a true real-time vintage database, because it does not model revisions, but it is materially stronger than using the final macro history with no delay assumption at all.

Fundamental inputs are handled more conservatively. The SEC quality signal is still a static cross-sectional research prior rather than a fully point-in-time fundamentals panel. For that reason the research engine supports a strict timing mode in which the SEC quality sleeve is disabled and macro inputs are forced to use the largest configured lag. The point of this design is not to pretend that point-in-time fundamentals are already solved, but rather to make the timing caveat measurable and visible in the evaluation workflow.

5 Methodology

5.1 Problem Formulation

Let $r_{t+1} \in \mathbb{R}^N$ denote next-period asset returns for a universe of N tradable assets. At time t, the system observes a feature state $x_t \in \mathcal{X}$ constructed from cross-sectional factor values, volatility forecasts, regime beliefs, macro inputs, recent portfolio returns, and allocation summaries. The alpha layer maps these features into an expected-return score vector $\alpha_t \in \mathbb{R}^N$ and a confidence vector $c_t \in \mathbb{R}^N$.

The control layer then chooses a portfolio allocation w_t and a cash allocation. The one-step objective is not written as a pure return-maximization problem. Instead, it is implicitly multi-objective: maximize long-run wealth while limiting drawdown, volatility, and unnecessary turnover. This is the reason the project separates alpha estimation from control: the two tasks operate on different time scales and carry different inductive biases. Importantly, the learned controller does not forecast r_{t+1} directly; it maps a state summarizing alpha opportunity, current allocation, and recent portfolio outcomes into exposure and overlay decisions.

5.2 Cross-Sectional Factor Model

The factor sleeve combines four z-scored components: momentum, a value proxy, a quality proxy, and a low-volatility tilt. Letting m_i, v_i, q_i, and ℓ_i denote the standardized scores for asset i, the composite factor score is

$$f_i = 0.30m_i + 0.25v_i + 0.25q_i + 0.20\ell_i.$$

This choice reflects a balanced, multi-style factor book rather than a single directional bet. In the revised architecture, this factor book is the dominant alpha source, with the other retained sleeves acting mainly as stabilizers or confidence modifiers rather than as separate return engines.

5.3 Volatility and Regime Models

Each name receives a GARCH(1,1) forecast

$$\sigma_t^2 = \omega + \alpha_1 \epsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2,$$

where $\omega > 0$, $\alpha_1 \geq 0$, and $\beta_1 \geq 0$ are the intercept, ARCH, and GARCH coefficients respectively. This conditional variance estimate is used for confidence scaling and risk awareness. Separately, a two-state HMM produces a bull-versus-bear regime belief. The final regime signal is

$$\text{RegimeBelief}_t = 0.75 \cdot \text{HMMBelief}_t + 0.25 \cdot \text{MacroBelief}_t.$$

5.4 Adaptive Alpha Combination

Let $s_{i,t}^{(k)}$ denote the signal from source k for asset i at time t. The expected return score is formed as

$$\alpha_{i,t} = \sum_k w_t^{(k)} s_{i,t}^{(k)},$$

where the source weights $w_t^{(k)}$ adapt over time based on recent realized signal quality. In the implementation, this is proxied by recent Spearman information coefficients between source scores and realized returns.

This adaptive weighting is important because the retained sleeves are not equally informative across all market states. In the revised architecture, however, adaptive weighting is explicitly treated as an ablation rather than as an article of faith. Fixed weights remain a strong baseline precisely because the archived bundle suggests that adaptive combination does not obviously add value in the current sample.

5.5 Constrained Allocator

The controller is not handed raw scores and asked to build a book from scratch. Instead, the revised architecture inserts a constrained intermediate allocator between the alpha layer and the control layer. The factor-anchored target book is

$$w_t^{\text{target}} = 0.90\, w_t^{\text{factor}} + 0.10\, w_t^{\text{stabilizer}},$$

where the stabilizer is a long-only minimum-variance-style portfolio estimated from a Ledoit–Wolf shrinkage covariance matrix.

The allocator solves a small regularized problem of the form

$$\min_w \quad \lambda_{\text{risk}} w^\top \Sigma_t w - \lambda_\alpha \alpha_t^\top w + \lambda_{\text{anchor}} \|w - w_t^{\text{target}}\|_2^2 + \lambda_{\text{turn}} \|w - w_{t-1}\|_2^2$$

subject to $w_i \geq 0$, $\sum_i w_i = 1$, per-asset caps, and group concentration limits. This intermediate layer is not only an architectural convenience; it reflects the practical fact that many real portfolio systems mediate between forecast signals and final positions through a constrained allocator.

5.6 Control Methods

The completed frozen-bundle compares five families of control methods built on the same finance-first alpha engine and constrained allocator.

1. **Alpha and fixed-rule baselines.** The suite includes `factor_only`, a fixed 95% allocator, volatility targeting, drawdown-based deleveraging, regime-aware rules, and simple rule ensembles.
2. **Contextual bandits.** Three lightweight bandit controllers are evaluated: LinUCB, Thompson sampling, and an ε-greedy linear controller.
3. **Supervised control.** An expanding-window classifier predicts the most attractive exposure bucket from the same compact state summary.
4. **Robust optimization.** A CVaR-aware robust allocator augments the constrained optimizer with downside-sensitive risk control and is the main non-learning advanced comparator.
5. **RL comparators.** Tabular portfolio Q-learning remains as a direct learned-controller baseline, and end-to-end PPO is retained as a monolithic comparator outside the main control-comparison suite.

5.7 Control as Risk-Shaping Optimization

The revised paper interprets portfolio control as a risk-shaping problem rather than as a pure forecasting problem. Let R^π denote the realized return distribution induced by controller π on top of the fixed alpha engine and constrained allocator. Then the generic objective is

$$\max_{\pi} \ \mathbb{E}[R^\pi] \quad \text{s.t.} \quad \mathcal{L}(R^\pi) \text{ is shaped toward acceptable downside behavior,}$$

or, equivalently, $\max_\pi \ \mathbb{E}[R^\pi] - \lambda \mathcal{L}(R^\pi)$ for an implicit risk functional $\mathcal{L}$. In that sense, all controllers can be viewed as different approximations to risk-shaping operators applied to the return distribution.

More formally, let $F_t^{(0)}$ denote the reference one-step return distribution generated by the fixed alpha engine and constrained allocator before a controller intervenes. Each controller then induces an operator $\mathcal{S}_\pi$ such that

$$F_t^\pi = \mathcal{S}_\pi\left(F_t^{(0)}\right) = (\mathcal{T}_\pi \circ \mathcal{Q}_\pi)\left(F_t^{(0)}\right),$$

where $\mathcal{Q}_\pi$ is a *selection operator* that changes portfolio weights, scenario weights, or exposure levels, and $\mathcal{T}_\pi$ is an *outcome operator* that reshapes realized payoffs after a portfolio has been selected. The controller search problem is therefore

$$\pi^\star = \arg\max_{\pi \in \Pi} \left\{ \mathbb{E}_{F_t^\pi}[R] - \lambda \mathcal{L}(F_t^\pi) \right\}.$$

This is the paper's central theoretical lens. Rules, bandits, RL, CMDP, CVaR, convexity, and council meta-control differ not mainly in prediction, but in the class of risk-shaping operators they apply to the same underlying return distribution. Rules, bandits, RL, and CMDP mainly approximate selection operators $\mathcal{Q}_\pi$; convexity is primarily an outcome operator $\mathcal{T}_\pi$; the council learns a mixture operator over expert policies; and the winning controller can be written compactly as $\mathcal{S}_{D+} = \mathcal{T}_{\text{convex}} \circ \mathcal{Q}_{\text{CVaR}}$.

This operator lens also connects the controller comparison to classical distortion-risk ideas. Dual-theory and distorted-expectation views interpret risk control as transforming probabilities or outcomes rather than

only changing point forecasts [21–23]. In that language, CVaR is a coherent left-tail distortion applied during portfolio selection, while the convexity layer is a discrete outcome distortion applied to realized returns. The framework therefore does more than label winners: it classifies controllers by how they reshape the induced return distribution.

The key empirical result is therefore not only that D+ wins, but that the winner composes two different risk-shaping operations. Most learning-based controllers in the bundle mainly act through a selection operator $\mathcal{Q}_\pi$. The convexity layer contributes through an outcome operator $\mathcal{T}_\pi$. The strongest controller is the first one in the study that combines both in a structured way.

Method family	**Risk-shaping interpretation**
Rules	Heuristic constraints on exposure and participation
Bandits	Contextual reward maximization under noisy feedback
RL	Sequential policy optimization over coarse actions
CMDP	Constrained policy optimization with a dual penalty
CVaR	Coherent left-tail risk minimization in portfolio selection
Convexity	Payoff transformation in stressed states
Meta-control	Mixture of policies with adaptive expert weights

5.8 CVaR-Robust Controller

Method D replaces scalar exposure scaling with a direct long-only weight optimizer built on the same constrained target book. Let μ_t denote the combined alpha scores scaled into daily-return units and let Σ_t denote the Ledoit–Wolf covariance estimate. The controller solves

$$\begin{aligned}\min_{w} \quad & \lambda_{\text{risk}} w^\top \Sigma_t w - \lambda_\alpha \mu_t^\top w + \lambda_{\text{anchor}} \|w - w_t^{\text{target}}\|_2^2 \\ & + \lambda_{\text{turn}} \|w - w_{t-1}\|_2^2 + \lambda_{\text{CVaR}} \, \text{CVaR}_{0.95}(L_t(w)),\end{aligned}$$

subject to the same long-only, sum-to-one, position-cap, and concentration constraints as the allocator. The loss term $L_t(w)$ is estimated from 500 Gaussian return scenarios whose mean is given by the current alpha scores and whose covariance is given by the shrinkage estimator. In implementation, λ_{CVaR} is increased when regime belief deteriorates and when recent drawdown exceeds 5%, so the robust controller becomes more conservative in stressed states without changing the underlying alpha engine.

5.9 Convexity-Aware Protection Modes

The D+ extension does not replace the robust optimizer. It post-processes the net portfolio return produced by the controller through a small three-mode payoff-shaping layer:

$$\tilde{r}_t = r_t^{\text{net}} - c(m_t) + \lambda_{m_t} \max(0, \tau - r_t^{\text{net}})^2,$$

where $m_t \in \{0, 1, 2\}$ corresponds to none, mild, or strong protection and $\tau = 0$ in the archived bundle. The daily carry schedule is $(0, 1.5, 4.0)$ bp and the quadratic coefficients are $(0, 1.5, 4.0)$. Strong protection activates whenever recent drawdown is at most -10%, realized annualized volatility is at least 26%, or regime belief falls below 0.30. Mild protection activates whenever drawdown is at most -5%, volatility is at least 18%, or regime belief falls below 0.45. Otherwise the mode is off. The layer therefore pays small carry in stressed states and only produces benefit on left-tail days, making D+ a stylized convex protection overlay rather than a separate options engine. The archived diagnostics record convexity mode frequencies, carry paid, and benefit realized by regime.

5.10 Logistic and MLP Meta-Control

Controller E is a soft council over three meaningful experts: A4 regime rules, B1 LinUCB, and D CVaR-robust. Each expert produces a candidate book $b_t^{(k)}$, and the council outputs

$$b_t = \sum_{k=1}^{3} \pi_k(x_t) b_t^{(k)}.$$

The gate state is intentionally small: alpha strength, recent drawdown, realized volatility, and regime belief. The gate is a multinomial logistic model retrained every 63 trading days once at least 40 labeled examples are available. Before retraining is possible, it starts from a 20/20/60 prior over regime rules, LinUCB, and CVaR-robust and tilts that prior toward CVaR under risk pressure and toward LinUCB when alpha strength is high. Labels are generated ex post from same-day expert net returns: the best realized expert becomes the target class for the next gate update. This keeps the council interpretable and lightweight: it is a supervised meta-controller over structured experts rather than a black-box RL policy over the full portfolio. E+ applies the same convexity layer after the blended council book is formed.

Candidate G keeps the same expert set and mixing equation but replaces the logistic gate with a small multilayer perceptron. Its gate state is a 16-dimensional environment feature vector built from trailing alpha quality, risk and volatility, market stress, regime, trend, and concentration statistics. The default hidden-layer layout is $(32, 16)$ with early stopping and ℓ_2 regularization. The MLP is retrained every 63 trading days once at least 60 labeled examples are available; before that point it falls back to the same heuristic prior used by the council. G+ applies the same convexity layer after the blended MLP-controlled book is formed. This pair isolates whether a more expressive gate improves on the lighter logistic council without changing the underlying experts.

5.11 CMDP-Lagrangian Control

Candidate F keeps the same compact discrete control problem as tabular Q-learning, but it modifies the learning objective to a constrained MDP. Its state uses the same discretized summary of alpha strength, recent drawdown, realized volatility, and regime belief, and the action ladder is the invested-fraction set $\{0.82, 0.90, 0.95, 0.98, 1.00\}$. The default constrained problem is

$$\max_{\pi} \ \mathbb{E}[R_t] \quad \text{s.t.} \quad \mathbb{E}[C_t] \leq \kappa,$$

where C_t is a drawdown cost and $\kappa = 0.12$ in the archived bundle. The implementation uses a Lagrangian relaxation,

$$\tilde{R}_t = R_t - \lambda_t(C_t - \kappa),$$
$$\lambda_{t+1} = \max(0, \lambda_t + \eta(C_t - \kappa)),$$

with $\lambda_0 = 1.0$ and $\eta = 0.05$. By default $C_t = \max(0, -\mathrm{DD}_{t+1})$, i.e. the positive part of next-step recent drawdown; an alternative tail-loss version replaces this with $\max(0, -r_t - \delta)$ using $\delta = 1\%$. The CMDP candidate therefore tests whether constrained learning can close the gap to robust optimization without changing state size or model capacity.

5.12 Portfolio RL

The revised portfolio RL state is deliberately minimal. It is constructed from the current alpha-score cross-section, the current portfolio-weight summary, and recent portfolio returns. This pruned state replaces the richer regime, uncertainty, volatility, and hedge-specific state variables used in the older architecture.

If the RL agent chooses invested fraction b_t and active overlay size τ_t, the final invested portfolio is approximately

$$w_t = b_t \left[(1 - \tau_t) w_t^{\text{target}} + \tau_t w_t^{\text{satellite}} \right].$$

The satellite sleeve is not a fully independent book; it tilts the factor target toward the strongest alpha names. This keeps the full pipeline close to the working factor engine while still letting RL modulate aggressiveness.

5.13 Reward Design

The revised architecture uses a Sortino-style reward by default. In compact form,

$$R_t^{\text{portfolio}} = \frac{r_t - \hat{\mu}_t}{\hat{\sigma}_t^{\text{down}} + \varepsilon},$$

where $\hat{\mu}_t$ is a running mean and $\hat{\sigma}_t^{\text{down}}$ is a running downside-deviation estimate. The agent is therefore rewarded for improving return relative to downside risk rather than for taking more total variance in absolute terms.

The research runner also supports reward-function ablations over differential-Sharpe, raw-return, Sortino, and mean-variance formulations. This matters because reward design is one of the least observable but most important degrees of freedom in RL-for-finance systems. Because reward specification changes the controller's operating point, it should be treated as a methodological degree of freedom rather than as a neutral implementation detail.

5.14 No-Trade Band and Transaction Costs

To reduce turnover drag, a rebalance band suppresses small changes in target weights. When only part of the portfolio is changed, the active weights are rescaled to preserve the desired capital budget.

The backtest includes a three-part execution-cost approximation. For each rebalance, the strategy pays a fixed basis-point fee on turnover, an additional penalty proportional to turnover times realized volatility, and a size penalty linked to the largest one-name weight change. In symbols,

$$\text{TC}_t = c_0 \cdot \text{turnover}_t + c_1 \cdot \text{turnover}_t \cdot \hat{\sigma}_t + c_2 \cdot \max_i |\Delta w_{i,t}|.$$

This is not a full execution simulator, but it pushes the daily allocator away from unrealistic churn and makes the rebalance band economically meaningful rather than purely cosmetic.

6 Experimental Design

The revised frozen-bundle used in the paper contains 3,269 daily observations from April 1, 2013 through March 27, 2026. The archived bundle reported here is the timestamped results folder `results/20260330_033844`. It contains legacy ablations, 19 control-method candidates across three train/test split settings, a strict-timing check, six rolling windows, four cost settings, three rebalance-band settings, three macro-lag settings, four reward settings, and five blocked time-series CV folds. The manuscript is tied to that explicit artifact bundle, especially `research_summary.json`, `research_control_comparison.csv`, and the companion CSV and PNG outputs. This does not eliminate prior exploratory iteration, but it does make the reported bundle explicit and reproducible. The codebase, manuscript sources, and artifact layout are maintained in the public repository https://github.com/ioan-hedea/quant-paper-lab, so the paper's frozen-bundle framing points to a concrete implementation record rather than to a private offline archive.

Within that archived suite, action ladders, optimizer settings, reward choices, and comparison baselines are held fixed across ablations, split-fraction tests, rolling-window robustness slices, and significance calculations. The control comparison itself uses train fractions {0.4, 0.5, 0.6} for nearly all methods and replaces the middle split with a 0.75/0.25 schedule for the Q-learning comparator to give the tabular learner a more favorable sample budget. The evaluated family now includes no-control exposure, simple rules, ensembles, contextual bandits, supervised control, CVaR-aware robust optimization, convexity-aware protection, council meta-control, MLP meta-control, CMDP-Lagrangian control, and RL under the same cost and timing discipline.

At each date, the revised pipeline executes the following sequence. First, factor, GARCH, HMM, macro, and optional SEC features are generated from the available data. These are combined into expected-return scores via either the adaptive or fixed combiner. The constrained allocator produces the target book. The chosen controller then decides how much of that book to express via invested fraction and active overlay size. For the robust and council extensions, an optional convexity mode can then apply a stylized carry cost and left-tail payoff shaping term before the final realized portfolio return is recorded. That realized return is finally used to update the value table or policy state when the controller is learned.

6.1 Benchmark Logic

The benchmark set is chosen to isolate different sources of value. SPY measures whether the pipeline adds value relative to standard passive equity exposure. The factor-only benchmark isolates the pure alpha layer without any controller overlay. Because the archived bundle already uses a broad 77-ticker universe, this factor benchmark is also informative about what the diversified opportunity set can do without learned control. Comparing the full pipeline against the factor-only benchmark is particularly important because it reveals whether learning is adding value as a controller or merely acting as a drag on a strong signal engine.

The newer comparison baselines sharpen that logic further. The volatility targeting baseline asks whether most of the observed improvement can be matched by simple inverse-vol scaling. The drawdown-based deleveraging baseline asks whether a threshold rule can replicate the control layer's protection effect. The end-to-end PPO baseline addresses the paper's most distinctive architectural question: if one hands the same state information to a monolithic policy, does it outperform a hybrid design in which finance-specific modules produce the target book and RL only controls how aggressively to express it? These baselines are deliberately strong because they represent the kinds of simple overlays a practitioner would try before accepting the complexity of RL.

7 Evaluation Metrics

The reporting suite includes annualized return, annualized volatility, Sharpe ratio, Sortino ratio, maximum drawdown, Calmar ratio, and return-distribution diagnostics including a daily 5% Value-at-Risk estimate.

If $\{r_t\}_{t=1}^{T}$ denotes daily portfolio returns and r_f is the annual risk-free rate, then the annualized Sharpe ratio is

$$\text{Sharpe} = \frac{252\,\bar{r} - r_f}{\sqrt{252}\,\hat{\sigma}(r) + \varepsilon}.$$

The Sortino ratio replaces total volatility with downside volatility, while the Calmar ratio scales annualized return by the magnitude of maximum drawdown. Each metric emphasizes a different aspect of the return distribution, which matters in this project because the controller is explicitly designed to reshape risk, not merely to maximize mean return. Table 3 focuses on the primary comparison metrics used in the main strategy table: return, volatility, Sharpe, maximum drawdown, and Calmar. Sortino and Value-at-Risk are

Table 3: Control-comparison summary across train/test split settings. All methods share the same finance-first alpha engine and constrained allocator. Annualized figures.

Controller	Return	Vol.	Sharpe	Max DD	Calmar
No control (alpha engine)	9.2%	13.4%	0.40	−19.8%	0.46
A1: Fixed	9.0%	12.7%	0.40	−18.6%	0.48
A2: Vol-target	6.9%	10.0%	0.34	−13.3%	0.51
A3: DD-Delever	6.6%	11.7%	0.25	−18.4%	0.36
A4: Regime rules	8.8%	11.9%	0.41	−16.1%	0.54
A5: Ensemble (mean)	7.4%	11.5%	0.32	−15.3%	0.48
A5: Ensemble (min)	6.6%	10.1%	0.30	−12.7%	0.51
B1: LinUCB	9.0%	12.3%	0.42	−18.0%	0.50
B2: Thompson	7.9%	12.3%	0.33	−18.6%	0.42
B3: Eps-greedy	8.4%	12.3%	0.37	−18.0%	0.46
C: Supervised	8.9%	12.5%	0.40	−19.2%	0.45
D: CVaR-robust	17.6%	17.3%	0.81	−26.4%	0.68
D+: CVaR + convexity	**17.8%**	**16.5%**	**0.85**	**−18.9%**	**0.94**
E: Council	12.9%	14.1%	0.64	−20.8%	0.60
E+: Council + convexity	11.7%	13.6%	0.57	−17.3%	0.67
G: MLP meta	12.8%	13.9%	0.65	−19.7%	0.63
G+: MLP meta + convexity	11.8%	13.4%	0.59	−17.5%	0.68
F: CMDP-Lagrangian	8.1%	12.3%	0.35	−18.4%	0.43
RL: Q-learning	6.2%	10.9%	0.23	−15.3%	0.41

retained as secondary diagnostics in the text, reward-ablation evidence, and archived research artifacts.

8 Results

This section first reports the new control-comparison headline, then uses the legacy ablation and full-pipeline diagnostics as secondary evidence about which sources of complexity were worth keeping. The key change relative to earlier drafts is that the paper no longer treats RL as the protagonist. The main question is which controller is strongest on top of the same finance-first alpha engine.

8.1 Main Result: Control Comparison

The main result is the completed 19-method control-comparison suite. Table 3 reports mean annualized performance across the three train/test split settings used in the frozen-bundle.

Comparison to Passive Benchmark (SPY). As an external calibration point, SPY averages 14.0% annualized return, 18.8% volatility, 0.56 Sharpe, and −30.6% maximum drawdown across the same three control-comparison splits. It is not shown as a row in Table 3 because SPY is a passive benchmark rather than a controller applied to the shared alpha engine. In that numerical sense, `D_plus_convexity` and plain `D_cvar_robust` are the only controllers in the study that clearly exceed SPY on both mean return and mean Sharpe, while the meta-control variants are competitive on Sharpe but not on mean return.

Among the learned methods, `G_mlp_meta` is the closest numerical SPY challenger on risk-adjusted terms: it gives up mean return (12.8% versus 14.0%) but improves Sharpe (0.65 versus 0.56) and materially reduces mean maximum drawdown (−19.7% versus −30.6%). That makes it a credible learned benchmark

Table 4: D+ versus the passive benchmark SPY across the same three control-comparison splits.

Metric	SPY	D+	Δ(D+–SPY)
Annualized return	14.0%	17.8%	+3.8 pp
Sharpe	0.56	0.85	+0.29
Max DD	−30.6%	−18.9%	+11.7 pp

even though it still trails the robust frontier.

Table 4 makes the passive comparison explicit. D+ achieves higher return and Sharpe than SPY while also reducing drawdown in the evaluated setting. The correct claim is not that the paper "beats SPY" in general, but that `D_plus_convexity` is the strongest point-estimate challenger to SPY in this frozen-bundle evaluation. The cached reference-split bootstrap against SPY is also supportive on Sharpe ($\Delta \approx +0.45$, 95% CI $[0.06, 0.82]$, $p = 0.015$), even though the return difference is not yet significant at that split. This SPY comparison is economic calibration rather than the paper's scientific contribution: the contribution is a principled comparison of control mechanisms under shared alpha, timing, and cost assumptions.

The main answer to **RQ1** is now more layered than in the earlier draft. Among the base controllers, CVaR-aware robust optimization remains the strongest non-extension method at 0.81 mean Sharpe. But the best overall point-estimate controller in the finished suite is `D_plus_convexity`, which improves the robust baseline to 0.85 Sharpe and 0.94 Calmar while also reducing mean maximum drawdown from -26.4% to -18.9%. The best single run in the control-comparison suite, `D_plus_convexity_tf0.50`, reaches 21.62% annualized return with 1.03 Sharpe.

The main answer to **RQ2** is also positive. Simple rules can provide competitive risk control. In particular, regime-aware rules reach Sharpe 0.415 with maximum drawdown -16.1%, improving on the no-control alpha engine's 0.402 Sharpe and -19.8% drawdown. Fixed allocation is similarly competitive, while vol-targeting offers the best drawdown control among the simple rules at the cost of lower return. No single method dominates every risk metric: if maximum drawdown is the binding constraint, the conservative rule families are still more attractive than the highest-return controllers.

The main answer to **RQ3** is negative for the learning-based methods. The learning stack does improve once it is reframed as meta-control rather than standalone RL: the MLP meta-controller reaches 0.65 Sharpe and the logistic council 0.64, both comfortably above the bandits, supervised controller, CMDP-Lagrangian controller, and Q-learning RL. But even those stronger learned extensions still trail both plain CVaR-robust at 0.81 and `D_plus_convexity` at 0.85. Adding convexity to the meta-controllers reduces drawdown modestly but gives back Sharpe, so the evidence favors learned gating over learned payoff shaping. LinUCB remains the best non-meta learner at 0.42 Sharpe, CMDP-Lagrangian reaches 0.35, and tabular Q-learning remains weak at 0.23 even after being given a more favorable 75/25 training split in the middle evaluation setting.

Read as best-in-class representatives, the finished comparison is internally coherent: A4 is the best simple rule, B1 the best non-meta learner, G the best learned controller overall, F the best constrained learner, D the best robust baseline, and D+ the best overall controller. Figure 2 makes the same pattern visual.

The MLP result is therefore not an isolated extra model. It is a boundary result. A more expressive learned gate does improve slightly on the logistic council (0.65 versus 0.64 Sharpe), which confirms that learning helps most at the meta-control layer. It is also the only learned controller numerically close to SPY, trailing on mean return but exceeding SPY on mean Sharpe and materially reducing drawdown. But that gain is still marginal relative to the jump from learned control to the robust stack, so expressive gating does not close the

Table 5: Pairwise significance for D+ versus key baselines at the reference split, including the strongest learned controller, using the cached block-bootstrap evaluation.

Comparator	ΔSharpe	95% CI	p-value
D: CVaR-robust	+0.12	[−0.08, 0.32]	0.490
A4: Regime rules	+0.45	[0.06, 0.90]	0.040
B1: LinUCB	+0.47	[0.11, 0.90]	0.010
G: MLP meta	+0.28	[0.00, 0.54]	0.045
SPY	+0.45	[0.06, 0.82]	0.015

gap to explicit risk shaping.

One important qualification is statistical rather than economic. The archived controller-to-controller bootstrap table compares plain `D_cvar_robust` against several alternatives at the reference split. Those tests show statistically significant annualized-return advantages versus the no-control alpha engine, regime rules, LinUCB, and supervised control, and statistically significant Sharpe improvements versus CMDP-Lagrangian and Q-learning RL. For the headline extension `D_plus_convexity`, Table 5 reports the main pairwise comparisons and Appendix Table 11 reports the complete reference-split table against every controller plus SPY.

Table 5 makes the current statistical picture more precise. The convexity-enhanced robust controller is not only the point-estimate leader, but it is also statistically superior to regime rules, LinUCB, the strongest learned controller `G_mlp_meta`, and SPY on Sharpe at the reference split under the cached block-bootstrap. Its edge over plain `D` remains economically positive but not statistically decisive on Sharpe. In the full reference-split table released with the artifact bundle, D+ is also statistically superior to the factor-only, fixed-allocation, vol-target, drawdown-deleveraging, and several other controller baselines on Sharpe. The appropriate claim is therefore stronger than in the previous draft: `D_plus_convexity` is the point-estimate leader with direct statistical support against the main non-CVaR baselines and against the best learned controller, while the residual open question is separation from plain `D_cvar_robust` itself.

The strongest practical insight from the new extensions is that convexity-aware payoff shaping directly addresses the earlier CVaR caveat. Plain CVaR-robust still has the worst mean maximum drawdown in Table 3 at -26.4%, which confirms that CVaR control and realized pathwise drawdown are not the same objective. But `D_plus_convexity` reduces that drawdown to -18.9% while improving both Sharpe and Calmar, making the convexity layer a meaningful refinement rather than a cosmetic add-on. The remaining Pareto tension is now between the strongest overall trade-off in `D+`, the cleanest simple-rule protection in A4 and A2, and the more moderate adaptive learning behavior of the meta-controller variants.

> **Why D+ wins.** (1) $\mathcal{Q}_{\text{CVaR}}$ improves tail-aware portfolio selection. (2) $\mathcal{T}_{\text{convex}}$ repairs the residual pathwise drawdown weakness of plain CVaR. (3) The composition preserves participation while reducing left-tail damage. (4) Learned controllers improve adaptivity, but not enough to replace explicit risk shaping.

Regime-conditioned performance. Reference-split regime slices clarify when each control style helps. During the 2020 crash window (2020-02-19 to 2020-04-30), `D+` produced +9.0% cumulative return with 1.04 Sharpe, -20.0% maximum drawdown, and mean invested fraction $b_t \approx 1.00$, versus plain `D` at -7.1%, -0.26, -31.2%, and $b_t \approx 1.00$, and SPY at -13.2%, -0.75, -33.7%, and $b_t = 1.00$. In the 2021 bull market, both `D` and `D+` stayed essentially fully invested and captured much of the upside; `D+` delivered +24.7% cumulative return with 1.58 Sharpe and -8.0% maximum drawdown, while SPY remained the strongest pure bull-market benchmark at +28.7%, 2.01, and -5.1%. In the 2022 bear market, the picture shifts: A4 regime

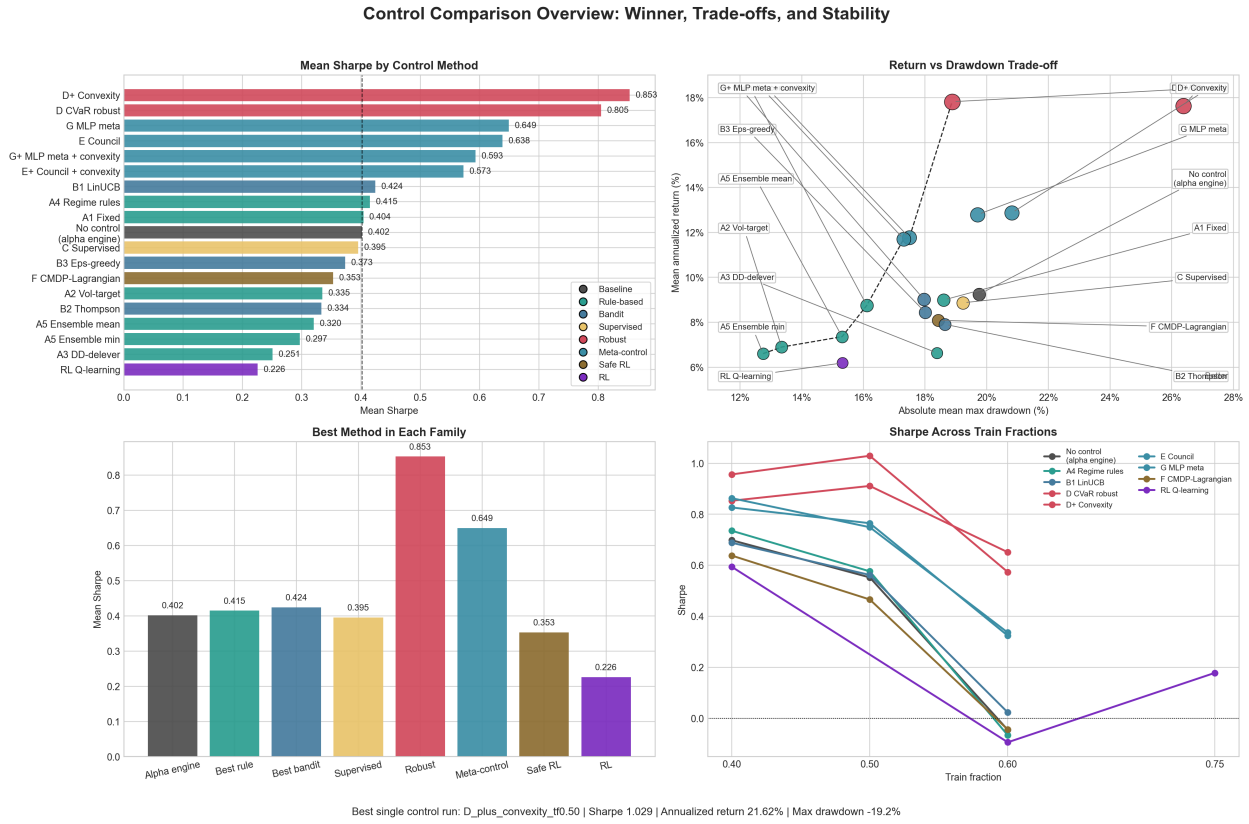

Figure 2: Control-comparison overview of the March 30 frozen-bundle. The four panels rank all controllers by mean Sharpe, summarize return–drawdown trade-offs, report the best method in each controller family, and show Sharpe stability across train fractions. `D+` leads overall, plain `D` is the strongest non-extension baseline, and the MLP meta-controller is the strongest learned challenger.

rules reduced mean invested fraction to 0.87 and achieved 0.5% cumulative return with -14.7% maximum drawdown, while `D+` stayed fully invested, lost 5.2%, and drew down 17.8%. These slices explain why no single method dominates every environment. D+ is strongest on the overall trade-off, but regime rules remain attractive when shallow realized drawdown is the binding objective.

Implication for the revised RQs. **RQ1**: answered in favor of the convexity-enhanced robust stack, with plain CVaR-robust as the strongest non-extension baseline. **RQ2**: answered positively, with regime-aware rules providing the strongest simple-rule baseline and vol-target providing the shallowest drawdowns. **RQ3**: answered negatively for learning-based control; even the learned meta-controllers remain below the best optimization-based method.

8.2 Legacy Ablation and Complexity Pruning

The control-comparison headline does not make the legacy ablation evidence irrelevant. On the contrary, the ablation table explains why the architecture was revised in the first place. It asks whether the old RL-heavy stack was actually earning its complexity before the broader controller comparison was run. The factor-only row in Table 6 is therefore the benchmark from the legacy ablation suite, not the control-comparison baseline from Table 3. The two tables summarize different archived suites: Table 3 compares controllers over the revised control-comparison design, whereas Table 6 summarizes older pruning runs whose purpose was to decide which stack components to keep. Mechanically, the legacy ablation uses the earlier pruning

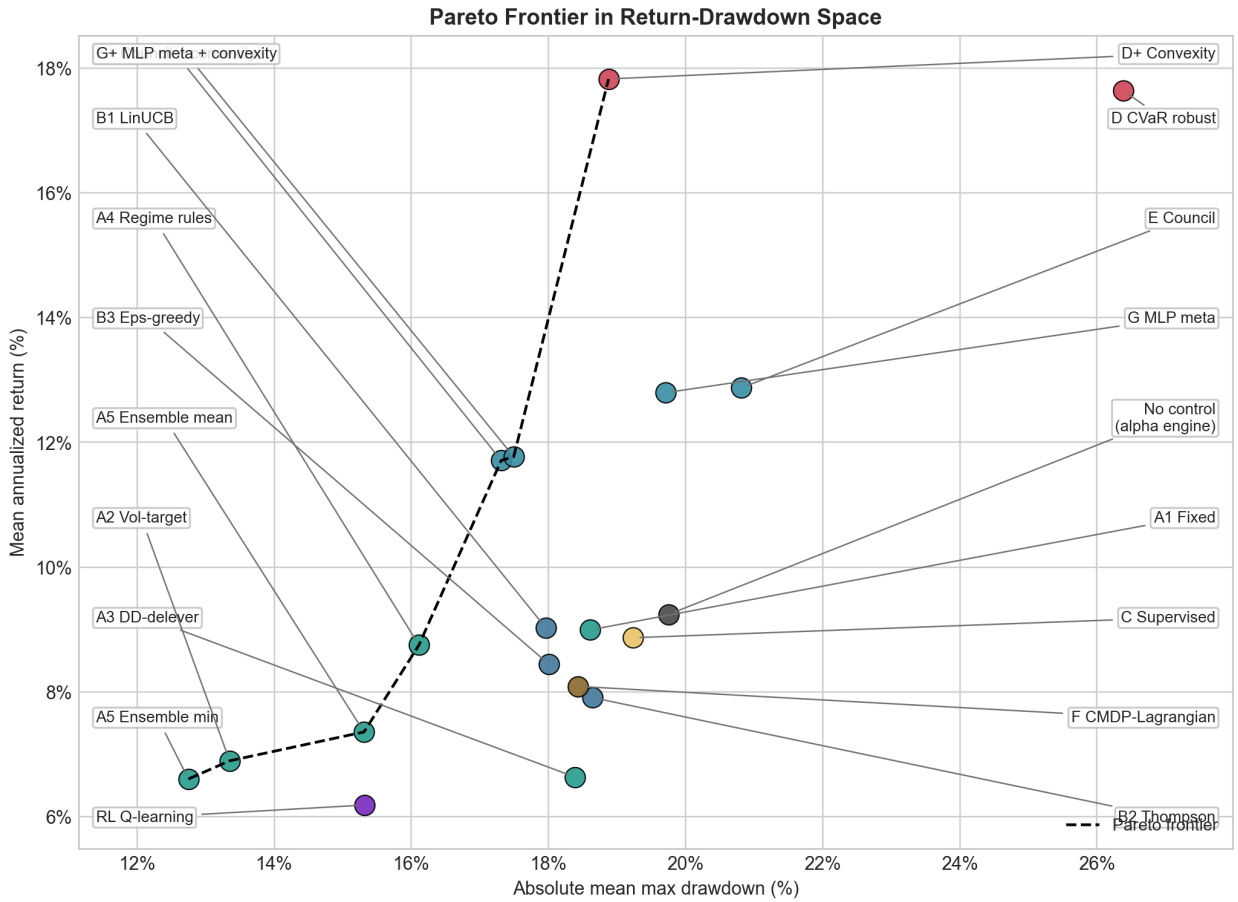

Figure 3: Pareto frontier in return–drawdown space for the finished 19-controller comparison. No single method dominates every metric. Plain `D_cvar_robust` delivers high return but deep drawdown, the simple-rule controllers anchor the conservative end of the frontier, and `D_plus_convexity` moves the robust optimizer toward a substantially better overall trade-off by keeping high return while materially reducing drawdown.

harness and its own evaluation schedule, so its 0.82 factor-only row reflects the factor core inside that older context rather than the no-control controller row from the revised comparison suite. In particular, the legacy factor-only row comes from a looser pruning-harness configuration, whereas the no-control row in Table 3 still passes through the revised constrained allocator with turnover penalties, long-only caps, and group limits, which compresses both return and volatility.

The ablation story remains sharply negative for the old learned stack. Averaged across split settings, the factor-only benchmark reaches mean Sharpe 0.82, whereas the legacy full pipeline reaches only 0.29. Fixed alpha combination is stronger than adaptive combination, and every additional learned layer beyond the factor core lowers average Sharpe materially. This is exactly the pruning result that motivated the shift from an RL-centric paper to a control-method comparison paper.

Implication for RQ3. The old RL-heavy complexity is not justified. The factor core is the credible source of alpha, and the original full pipeline is best interpreted as a negative control that motivated the revised study.

Table 6: Ablation summary across train/test split settings in the legacy pruning suite. These rows come from a different archived evaluation context than Table 3 and should not be compared numerically line by line.

Component	Return	Vol.	Sharpe	Max DD	Calmar
Factor-only benchmark	20.2%	19.8%	0.82	−27.6%	0.74
Alpha stack (fixed weights)	9.9%	13.4%	0.45	−19.4%	0.51
Alpha stack (adaptive)	9.2%	13.4%	0.40	−19.6%	0.46
Portfolio RL (fixed alpha)	7.8%	12.1%	0.32	−17.5%	0.44
Full pipeline (fixed alpha)	7.9%	12.2%	0.33	−18.1%	0.43
Full pipeline	7.3%	12.0%	0.29	−17.4%	0.42

Table 7: Rolling-window robustness summary for the full pipeline.

Statistic	Value
Rolling windows	6
Median full-pipeline Sharpe	0.59
Median full-pipeline Calmar	1.07
Median full-pipeline Max DD	−8.9%
Frac. windows full beats factor-only benchmark on drawdown	67%
Frac. windows full beats factor-only benchmark on Calmar	17%
Frac. windows full beats SPY on Sharpe	17%
Frac. windows full beats SPY on Calmar	17%

8.3 Legacy Full-Pipeline Reference and Significance

The old full pipeline is retained as a secondary reference because it anchors the transition from the earlier project narrative to the new one. In the refreshed timestamped confidence-interval artifact, its point estimates are 9.0% annualized return, 0.423 Sharpe, 0.459 Calmar, and −19.6% maximum drawdown.

The bootstrap significance summary in Table 8 shows that this legacy full pipeline is significantly worse than the factor-only benchmark on Sharpe and annualized return, significantly worse than vol-targeting on both dimensions, and materially weaker than drawdown-deleveraging on annualized return. It is not statistically distinguishable from SPY or end-to-end PPO on Sharpe in the completed bundle.

The Jobson–Korkie summary tells the same story more narrowly: the legacy full pipeline is significantly worse than the factor benchmark on Sharpe at the 5% level, while differences versus SPY, PPO, and the stronger rule baselines are not statistically decisive. So even as a legacy reference, the old RL branch does not recover the central economic claim.

8.4 Reward Sensitivity and Robustness of the Legacy Branch

Reward design affects where the retained full-pipeline controller lands on the risk-return frontier, but only within a relatively narrow band. Across the reward-function ablation, the Sortino reward is best for the legacy branch, reaching about 9.1% annualized return and 0.44 Sharpe, while the broader ranking versus factor-only, rules, and robust optimization is unchanged.

The rolling-window and time-series cross-validation diagnostics remain useful as appendix diagnostics for the retained legacy branch, not as main sources of evidence for the revised control-comparison claim. The rolling-window summary shows median full-pipeline Sharpe 0.59, median Calmar 1.09, and median

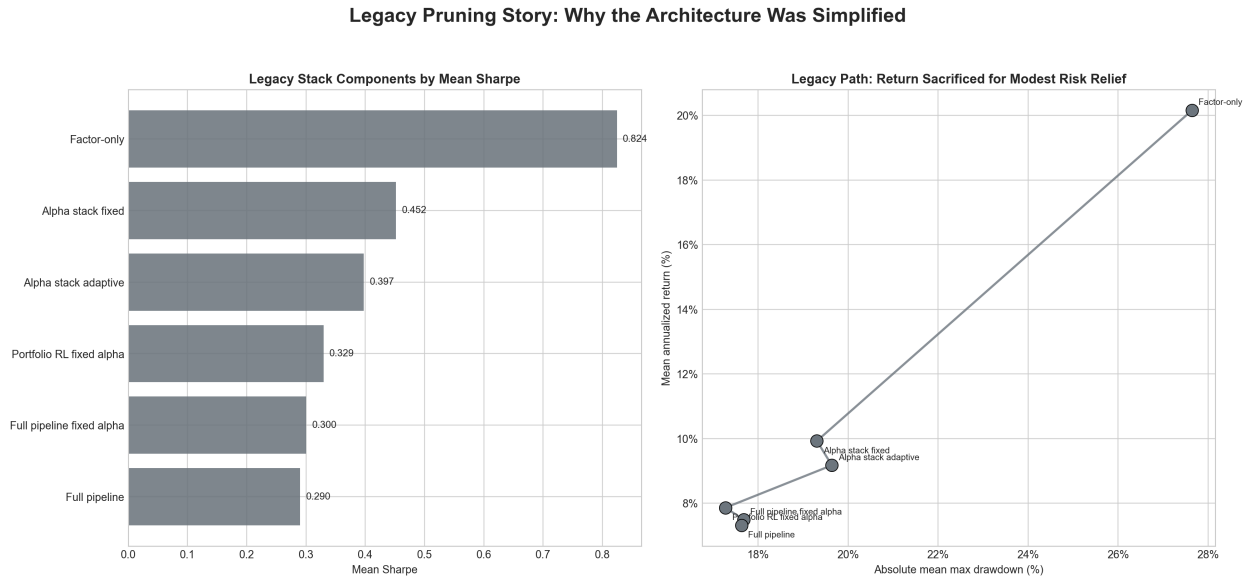

Figure 4: Legacy pruning story from the retained ablation suite. The left panel orders the old architecture variants by mean Sharpe; the right panel shows the pruning path in return-versus-drawdown space. Its role in the revised paper is diagnostic: it explains why the study moved away from defending the old RL stack and toward a broader control-method comparison.

Table 8: Block-bootstrap significance for the legacy full pipeline versus selected baselines (Sharpe deltas).

Comparison	Delta Sharpe	95% CI	p-value
Full – SPY	-0.16	[-0.91, 0.51]	0.615
Full – Factor-only benchmark	-0.58	[-1.14, -0.15]	0.005
Full – Vol-target	-0.47	[-0.98, -0.03]	0.035
Full – DD-Delever	-0.50	[-1.04, 0.06]	0.070
Full – E2E RL (PPO)	-0.14	[-1.19, 0.81]	0.820

maximum drawdown -8.9%. The corresponding benchmark-beat fractions remain weak: the legacy full pipeline beats SPY on Sharpe in only 17% of windows and beats the factor benchmark on Calmar in only 17% of windows. The blocked time-series CV summary is similarly modest, with Sharpe 0.41 ± 0.57 and Calmar 0.72 ± 0.51 across five folds.

Implication for the overall claim. The legacy branch still does not show a structural edge. Its main value in the revised paper is as a transparent negative result that motivated the broader controller comparison.

9 Discussion

The discussion can now be stated more directly. The paper's main contribution is a risk-shaping framework for portfolio control, not merely a 19-controller leaderboard. Under a shared alpha engine and shared timing/cost protocol, the winning design combines tail-aware selection of portfolio outcomes through CVaR with tail-aware transformation of realized payoffs through convex protection. The remaining controller comparisons then act as boundary results around that central mechanism.

The experimental results support three conclusions of methodological importance.

Table 9: Reward-function ablation for the full pipeline on the revised bundle.

Reward	Return	Sharpe	Max DD	Calmar
Differential Sharpe	9.3%	0.45	−17.0%	0.55
Return	9.5%	0.48	−17.2%	0.55
Sortino	9.5%	0.48	−17.0%	0.56
Mean-Variance	9.2%	0.45	−17.8%	0.52

First, the control-mechanism comparison has a strongest point-estimate winner. `D_plus_convexity` now leads the finished suite, reaching mean Sharpe 0.85 and Calmar 0.94 versus 0.40 for the no-control alpha engine, 0.42 for the best non-meta learner LinUCB, and 0.65 for the strongest learned controller `G_mlp_meta`. In other words, once the factor, GARCH, and HMM alpha engine is fixed, the biggest incremental value comes from explicit downside-aware optimization refined by payoff shaping rather than from additional sequential-learning complexity. The layered interpretation also matters: plain `D_cvar_robust` remains the strongest non-extension baseline, while the convexity layer materially improves its drawdown profile.

Second, simple rules are stronger than a complexity-first story would suggest. Regime-aware rules reach 0.415 mean Sharpe with better maximum drawdown than the no-control alpha engine, while fixed exposure and volatility targeting provide clean, interpretable trade-offs between participation and protection. This is a useful empirical result in its own right: the paper does not need learned control to justify itself, because a disciplined comparison already shows that simple rules can be credible, competitive baselines.

Third, the learning-based methods are mixed rather than dominant. LinUCB is the best non-meta learner and does slightly improve on the no-control alpha engine, but the bandits, supervised controller, CMDP-Lagrangian controller, and Q-learning RL all remain well below the robust stack on risk-adjusted performance. The positive exception is learned meta-control: `G_mlp_meta` reaches 0.65 Sharpe and `E_council` 0.64, while their convexity-augmented variants give back Sharpe in exchange for somewhat shallower drawdowns. That makes meta-control a plausible refinement direction even though it still does not unseat `D+`. That negative result is not an embarrassment; it is part of the methodological contribution. The point of the revised paper is to delimit where learning helps and where simpler or more structured controllers remain stronger. Learning contributes most when it is used to arbitrate among already-credible structured controllers rather than to replace financial structure end to end.

What did not work is therefore just as informative as what did. The old RL-heavy stack still collapses in the legacy ablation table. Drawdown-based deleveraging is weaker than the better simple rules. Ensemble rules are more stable than exciting. CMDP-Lagrangian control does not bridge the gap between optimization and learning, and tabular RL remains weak even after being given a more favorable split schedule. These failures are precisely what turn the project from a prototype report into a more credible control-method study.

9.1 Mechanism of CVaR + Convexity

Performance improvements arise not from better prediction, but from better shaping of the return distribution's left tail. Variance treats upside and downside symmetrically, whereas the practical objective in this study is to preserve participation while avoiding clustered losses. CVaR is better aligned with that asymmetry because it directly penalizes bad outcomes rather than average fluctuation.

The two layers operate on different parts of the same problem. CVaR acts on *which outcomes are selected*: it changes the optimized portfolio weights so that extreme-loss scenarios receive larger penalties during

portfolio construction. Convexity acts on *how outcomes are realized*: it post-processes realized returns by paying bounded carry in quiet states and adding nonlinear benefit only when realized outcomes fall into the left tail. The combination is therefore more expressive than either component alone: selection of safer portfolios is complemented by payoff reshaping in the states where plain reweighting still leaves pathwise losses exposed.

The convexity layer then adds selective protection rather than constant hedging. In the reference D+ split, the protection mode was inactive on about 41% of trading days, mild on about 36%, and strong on about 23%. Average daily carry was roughly 1.45 bps, average daily payoff benefit was about 1.89 bps, and the mean net adjustment remained positive. That selectivity is exactly what shows up in the crash slice: plain `D` lost 7.1% with -31.2% drawdown in the 2020 crash window, whereas `D+` gained 9.0% with -20.0% drawdown over the same dates.

The regime behavior also suggests that the mechanism is not simply binary de-risking. Both `D` and `D+` stay essentially fully invested in the crash, bull, and bear reference windows. The improvement therefore comes mainly from scenario-aware reweighting and left-tail payoff shaping, not from switching the book off. In that sense, CVaR behaves like a continuous regime-aware controller: it becomes more loss-averse in stressed states while remaining highly participatory in calm ones.

Finally, the robust stack is smoother than the discrete alternatives. Rules impose thresholds, bandits and RL switch among coarse actions under noisy signals, and meta-control only partially softens that problem. In the interpretability diagnostic, the MLP meta-controller still assigns the largest share to CVaR-robust through most of the reference split while redistributing the remaining mass between regime rules and LinUCB as conditions change. Even with meta-control and constrained learning extensions, the structured robust optimization approach remains dominant.

Proposition 1 (tail-shaping improvement under mild regularity). Let R be a one-step portfolio return with continuous distribution supported on $[r_{\min}, r_{\max}]$, let $q_\alpha := q_\alpha(R) < \tau$ denote the lower α-quantile, and define

$$g_m(r) = r - c(m) + \lambda_m(\tau - r)_+^2.$$

Assume (i) g_m is nondecreasing on $[r_{\min}, \tau]$, equivalently $2\lambda_m(\tau - r_{\min}) \le 1$, and (ii) $c(m) \le \lambda_m(\tau - q_\alpha)^2$. Then

$$\begin{aligned} \mathbb{E}[g_m(R)] &= \mathbb{E}[R] - c(m) + \lambda_m \mathbb{E}\left[(\tau - R)_+^2\right], \\ \mathrm{CVaR}_\alpha(-g_m(R)) &\le \mathrm{CVaR}_\alpha(-R) - \delta_\alpha(m), \end{aligned}$$

where $\delta_\alpha(m) := \lambda_m(\tau - q_\alpha)^2 - c(m) \ge 0$.

Proof. Let $A_\alpha := \{R \le q_\alpha\}$. Because $q_\alpha < \tau$, for every $r \in A_\alpha$ we have

$$g_m(r) - r \ge -c(m) + \lambda_m(\tau - q_\alpha)^2 = \delta_\alpha(m).$$

Assumption (i) preserves the ordering of tail states on $[r_{\min}, \tau]$, so the worst α-tail of $g_m(R)$ is still attained on A_α. Therefore

$$\mathrm{CVaR}_\alpha(-g_m(R)) = -\mathbb{E}[g_m(R) \mid A_\alpha] \le -\mathbb{E}[R \mid A_\alpha] - \delta_\alpha(m) = \mathrm{CVaR}_\alpha(-R) - \delta_\alpha(m).$$

The expectation identity follows by expanding $g_m(R)$ pointwise and taking expectations. Thus convexity reduces lower-tail risk whenever the protected-tail gain dominates carry. □

In the reference D+ split, $\tau = 0$, the strong-mode coefficient is $\lambda_m = 4$, and the observed raw-return minimum is about -11.3%, so assumption (i) implies $2\lambda_m(\tau - r_{\min}) \approx 0.90 < 1$ and is satisfied in the reported sample.

The key mechanism is now explicit: $g_m(r) \approx r - c(m)$ for ordinary states with $r \geq \tau$, while stressed states with $r < \tau$ receive a state-dependent upward adjustment. Convexity therefore improves performance by reshaping the left tail of the return distribution, not by improving prediction.

9.2 Regime-Dependent Controller Effectiveness

The results are consistent with a regime-dependent optimality hypothesis. Controllers differ primarily in how they respond to tail risk, so their relative ranking changes across bull, neutral, bear, and crisis states. Light-control methods remain attractive in benign markets, simple rules and bandits become competitive in quieter transitional states, plain CVaR improves when losses are more persistent, and D+ is strongest in the true crisis slice where both scenario selection and payoff reshaping matter at once.

This hypothesis helps organize the empirical evidence. In the 2021 bull market, SPY remains the cleanest upside benchmark and the lighter controllers do not need to overreact. In the 2022 bear period, regime rules remain attractive when the binding objective is shallower realized drawdown. In the 2020 crash, however, D+ is the standout because it combines crisis-aware reweighting with a selective convex response. Controller dominance is therefore conditional, not universal: methods differ mainly in how aggressively and smoothly they reshape the left tail.

9.3 Meta-Control as Mixture-of-Policies

The council and MLP meta-controller are best understood as mixture-of-policies models,

$$b_t = \sum_k \pi_k(x_t) b_t^{(k)},$$

which approximates lightweight model averaging over interpretable experts. This reduces variance relative to relying on a single learned controller and helps explain why meta-control is the strongest learned family in the paper. The MLP gate analysis makes the mechanism more concrete. In the reference split, the CVaR weight has a +0.29 correlation with realized stress, the LinUCB weight a -0.32 correlation, and aggregate gate shifts a +0.61 correlation with regime-belief switches. The gate therefore reacts most around regime transitions by modestly leaning further toward robust control rather than by collapsing to a one-expert policy.

At the same time, the meta-controllers still remain below D+. That gap is informative. Variance reduction alone is not enough to match direct distributional shaping. The logistic council and the MLP gate can learn when to lean more heavily on CVaR, rules, or LinUCB, and the MLP gate slightly improves on the logistic council (0.65 versus 0.64 mean Sharpe), but neither changes the underlying payoff map as directly as D+. In that sense, meta-control is a useful refinement layer rather than a replacement for the robust stack. A cached ex-post 50/25/25 blend of plain D, A4, and B1 return streams reaches about 0.92 mean Sharpe across the three split settings, which is above the 0.64–0.65 range of the learned gates in Table 3. So the present learned gates are valuable as boundary results, but they do not yet outperform a strong static mixture of the same structured experts. That in turn sets up the broader control-versus-learning conclusion: learning helps most when it arbitrates among strong structured policies, not when it is asked to replace explicit risk shaping.

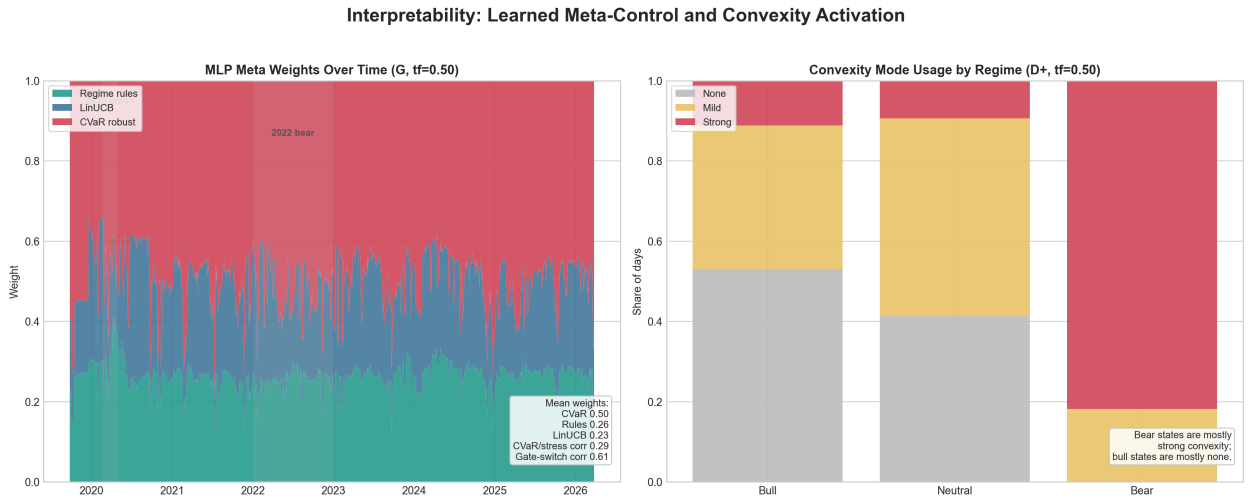

Figure 5: Interpretability diagnostics for the revised extensions. Left: shaded 2020-crash and 2022-bear windows show that the reference-split MLP meta-controller remains centered on `D_cvar_robust` while adjusting most around regime transitions. Right: convexity mode usage for `D_plus_convexity` shifts from mostly `none` in bull states to mostly `strong` in bear states. Together the panels support the paper's headline mechanism claim: the gains come from state-dependent distribution shaping and expert selection, not from a new predictive signal.

9.4 Control Versus Learning in Noisy Environments

In low signal-to-noise environments, structured risk-aware optimization appears more reliable than learning-based control. The revised bundle makes that contrast unusually clean because every method sits on top of the same factor/GARCH/HMM alpha engine. The primary contribution is therefore the convexity-enhanced robust controller `D+`; the council and MLP meta-controller are meta-control attempts that learn how to mix interpretable experts; and the CMDP controller is a constrained-learning bridge that tests whether adding a dual variable closes the gap between optimization and RL.

The answer from the finished bundle is no. Meta-control is genuinely interesting, because it improves materially over the standalone learners and the MLP gate is now the best learned method in the paper. But even that result still sits below `D+`, and CMDP-Lagrangian control remains much closer to the weaker learning baselines than to the robust frontier. That pattern suggests that, for daily portfolio rebalancing under noisy observations, directly encoding downside structure is more reliable than trying to learn it from the realized return stream. In other words, learning is most useful here as meta-control, not as primary control.

Mechanism diagnostic. Figure 6 gives the cleanest direct evidence for the mechanism claim. Relative to both factor-only and plain CVaR, D+ shifts the empirical left tail inward at the reference split: the worst-return mass is smaller and the 5% tail summary improves, even though mean return remains close to plain D. That pattern is exactly what the risk-shaping framework predicts. The improvement is not primarily a forecasting gain; it is a change in how bad outcomes are selected and then transformed.

9.5 When Practitioners Should Use Which Control Style

The archived evidence suggests three practical regimes of use. If the main objective is to maximize risk-adjusted performance on top of an already strong factor engine, the current bundle points first toward `D_plus_convexity` and second toward plain `D_cvar_robust`. The former is the strongest overall point-estimate trade-off in the completed comparison, while the latter is the strongest non-extension baseline.

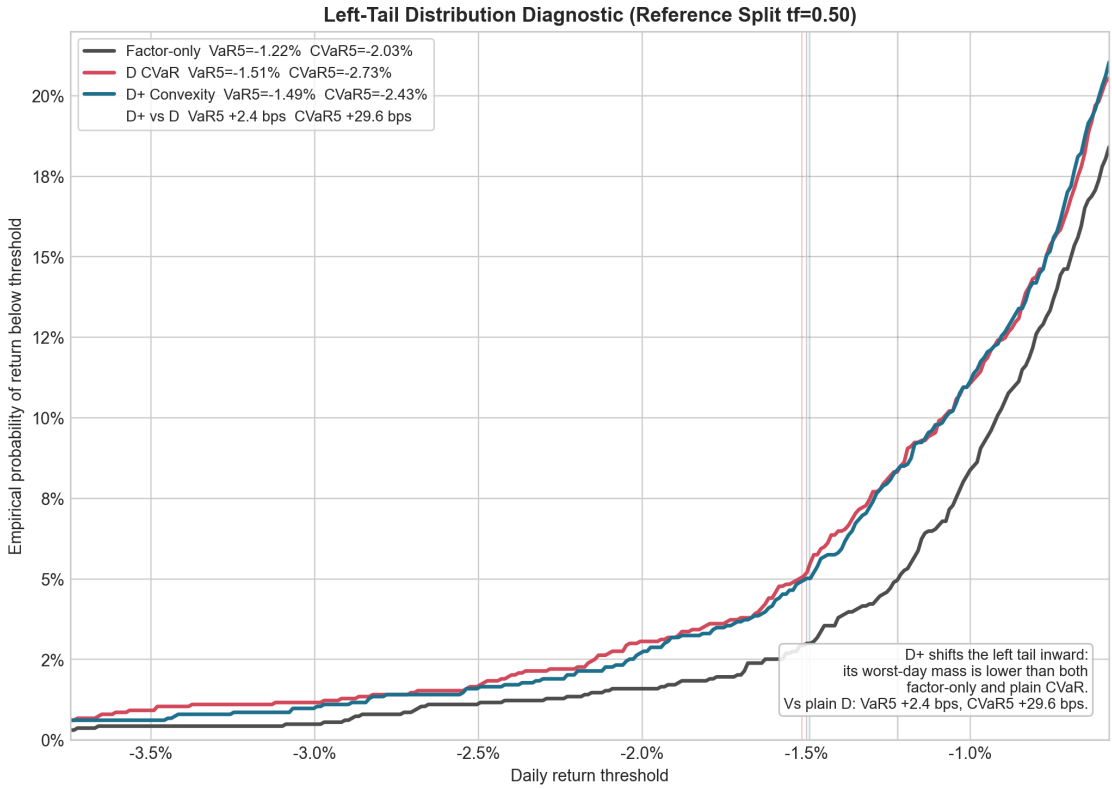

Figure 6: Left-tail empirical distribution diagnostic for the reference split. Compared with factor-only and plain CVaR, `D+` shifts the worst-return mass inward and improves the 5% tail summary while leaving average return roughly unchanged relative to plain `D`. The figure supports the paper's main mechanism claim: the improvement of `D+` is driven primarily by tail-risk reduction rather than by higher average returns alone. At the reference split, D+ improves 5% CVaR by about 30 bps versus plain D.

If simplicity, interpretability, and implementation speed matter more than squeezing out every last point estimate, regime-aware rules are the strongest practical baseline. They improve drawdown materially, remain easy to explain, and do not require the extra tuning burden associated with bandits or RL.

The learning-based methods are still useful as research instruments, but not as the default recommendation from this bundle. The MLP meta-controller suggests that lightweight learned meta-selection can be genuinely competitive, yet it still does not close the gap to the best robust-optimization stack. The practical contribution is therefore not "always use learning." It is to compare structured rules, learning, robust optimization, and payoff-shaping extensions over a shared alpha engine before accepting additional controller complexity.

10 Threats to Validity and Future Work

The implementation remains research code, and several caveats apply. Macro inputs are shifted by a configurable lag, but they are not full vintage series with revision-aware histories. SEC enrichment can be used as a static cross-sectional prior, but it still does not constitute a true point-in-time fundamentals database. Transaction costs are more realistic than before, yet they remain stylized rather than venue-specific. The robustness engine still depends on one underlying historical sample even when it slices that sample in multiple ways.

Several additional threats deserve explicit acknowledgement. First, the design has been iterated by

observing backtest behavior, which raises the possibility of researcher degrees of freedom even when each individual change is motivated. Second, the traded universe is broader than in the earlier prototype but still narrow relative to institutional equity universes, so the results may depend partly on the selected names. Third, daily data are appropriate for a medium- horizon allocation study but cannot validate intraday execution claims. Finally, the project deliberately leans on free or public data sources, which improves accessibility but necessarily limits point-in-time fidelity and market microstructure realism.

A further caveat is benchmark fairness: although the rule-based and PPO baselines are designed to be strong and informative, their exact specification still influences comparative conclusions. Because the evaluation suite reports many metrics and comparator strategies, some apparent wins and losses should also be interpreted with the usual caution around multiple comparisons. External validity is also limited: conclusions drawn from a daily U.S.-equity allocation setting may not transfer unchanged to other asset classes, frequencies, or market microstructure regimes.

The currently reported robustness summary is still based on only six rolling windows, five blocked time-series CV folds, and one underlying historical sample. The workshop interpretation should therefore be that the robustness engine is now part of the methodology, not that robustness has been exhaustively established across many independent market environments. At the reference split, the D+ significance story is now complete rather than partial: the artifact bundle includes 5,000-sample block-bootstrap tables for `D_plus_convexity` versus every controller baseline plus SPY, and those tests show direct Sharpe separation for D+ versus factor-only, regime rules, LinUCB, `G_mlp_meta`, and SPY, but not versus plain `D`. What remains future work is an aggregated multi-split or multi-window significance analysis, together with equally complete tables for the broader meta-controller family.

Natural next steps follow directly from this new result. The highest-value extensions are broader universes and asset classes, stricter point-in-time data treatment, aggregated multi-split significance, more systematic benchmark tuning, and deeper analysis of when council-style meta-control or convexity modes are activated. If more advanced learned controllers re-enter the stack later, they should do so as separately validated comparators rather than as assumed core architecture.

11 Conclusion

This paper reframes the project from an RL question into a broader empirical comparison of control mechanisms for factor-based portfolio management. The shared finance-first alpha engine remains modular and interpretable; the main scientific question is which controller adds the most value once that engine is already in place.

The completed frozen-bundle gives a clear answer. The strongest overall point-estimate controller is the convexity-enhanced robust stack `D_plus_convexity`; simple regime-aware rules are the best practical low-complexity baseline; and the learning-based methods, including CMDP and tabular RL, do not beat the best optimization-based approach. D+ is also the strongest point-estimate challenger to SPY in the completed bundle, while reference-split block-bootstrap already shows Sharpe separation versus factor-only, regime rules, LinUCB, and SPY, but not yet decisive Sharpe separation versus plain CVaR-robust. Equally importantly, the paper leaves behind a frozen-bundle evaluation protocol that can support future controller comparisons under a consistent empirical standard. In the reported sample, the evidence supports a stronger conceptual claim than the earlier draft: portfolio control is best understood here as risk shaping. Explicitly distribution-aware optimization refined by convexity-aware payoff shaping is the strongest current control layer, while the MLP gate marks the strongest learned challenger and a clear boundary result. The paper does not claim that D+ is universally optimal; it claims that under a fixed finance-first alpha engine and shared

experimental protocol, explicit risk shaping dominates the learned alternatives tested here.

A Implementation Snapshot

A.1 Revised-Study Snapshot

The revised frozen-bundle reported in the paper corresponds to the simplified finance-first stack summarized in Table 10. The most important change relative to the earlier project versions is that the paper now evaluates a family of controllers on top of the same allocator rather than trying to defend one learned controller in isolation.

Table 10: Active study design in the revised frozen-bundle.

Layer	Active design
Alpha layer	Factor, GARCH, and HMM sleeves only
Combiner	Adaptive IC-weighted combiner with fixed-weight ablation
Allocator	Long-only, capped, turnover-penalized, factor-anchored target book
Controller candidates	No control, simple rules, ensembles, contextual bandits, supervised control, CVaR-robust, CVaR + convexity, council meta-control, council + convexity, MLP meta-control, MLP meta-control + convexity, CMDP-Lagrangian, and Q-learning RL
Execution	No-trade band plus three-part transaction-cost model and optional convexity-mode carry term
Legacy reference branch	RL-heavy full pipeline, PPO comparator, and pruning ablations retained as negative-reference evidence

A.2 Research Artifacts

For the revised timestamped bundle discussed in the paper, the research runner writes the artifacts below into `results/20260330_033844/`:

1. `research_metrics.csv`: tabular output for ablation and robustness runs;
2. `research_control_comparison.csv`: split-aggregated controller comparison across the 19 evaluated methods;
3. `research_ablation_summary.csv`: split-aggregated ablation summaries;
4. `research_robustness_summary.csv`: rolling-window medians and benchmark beat fractions;
5. `research_regime_summary.csv`: regime-conditioned action, turnover, and return summaries;
6. `research_rolling_references.csv`: rolling benchmark metrics for SPY and the factor-only benchmark;
7. `research_bootstrap_cis.csv`: block-bootstrap confidence intervals for key strategy metrics;

8. `research_bootstrap_significance.csv`: legacy-full-pipeline bootstrap deltas versus benchmark strategies;
9. `research_control_significance.csv`: complete reference-split D+-versus-controller bootstrap deltas, including SPY;
10. `research_control_significance_full.csv`: full controller-significance table for the current bundle;
11. `research_control_significance_dplus.csv`: D+-centered slice of that table for direct manuscript use;
12. `research_jobson_korkie.csv`: parametric Sharpe-ratio comparison tests;
13. `research_ts_cv.csv`: blocked time-series cross-validation summaries;
14. `research_meta_blend_sanity.csv`: cached 50/25/25 fixed-blend sanity check versus the learned meta-controllers;
15. `research_mlp_gate_diagnostics.csv`: MLP gate stress, switch, and regime diagnostics;
16. `research_summary.json`: serialized configuration and best-per-suite summaries;
17. `control_method_overview.png`: main paper figure summarizing controller ranking, trade-offs, and split stability;
18. `control_pareto_frontier.png`: Pareto-frontier view of return versus drawdown;
19. `control_interpretability.png`: MLP gate and convexity activation diagnostic figure;
20. `control_tail_diagnostic.png`: left-tail diagnostic comparing factor-only, plain D, and D+;
21. `pipeline_rolling_windows.png`: appendix-only rolling-window diagnostic for the legacy branch; and
22. `pipeline_reward_ablation.png`: reward-ablation figure.

Together with the feature-availability contract embedded in each backtest run, these artifacts make it easier to compare results across alternative timing, cost, and control assumptions without manually reconstructing the experimental context after the fact.

A.3 Supplementary Diagnostic Figures

These figures are diagnostic and illustrative, but the primary evidentiary claims of the paper now rest on Tables 3–8 and Figures 2, 3, 4, 5, and 6. The appendix intentionally keeps the rolling-window figure as a legacy robustness supplement rather than a main-text result.

Table 11: Complete reference-split pairwise significance table for D+ versus every controller plus SPY (Sharpe deltas).

Comparator	**ΔSharpe**	**95% CI**	**p-value**
No control (alpha engine)	+0.48	[+0.13, +0.90]	0.025
A1: Fixed	+0.48	[+0.08, +0.88]	0.015
A2: Vol-target	+0.59	[+0.17, +1.08]	0.000
A3: DD-Delever	+0.67	[+0.20, +1.16]	0.000
A4: Regime rules	+0.45	[+0.06, +0.90]	0.040
A5: Ensemble (mean)	+0.56	[+0.16, +1.01]	0.005
A5: Ensemble (min)	+0.63	[+0.19, +1.10]	0.005
B1: LinUCB	+0.47	[+0.11, +0.90]	0.010
B2: Thompson	+0.58	[+0.15, +0.99]	0.000
B3: Eps-greedy	+0.51	[+0.13, +0.91]	0.010
C: Supervised	+0.47	[+0.08, +0.84]	0.025
D: CVaR-robust	+0.12	[-0.08, +0.32]	0.490
E: Council	+0.26	[-0.00, +0.49]	0.060
E+: Council + convexity	+0.25	[+0.05, +0.45]	0.025
G: MLP meta	+0.28	[+0.00, +0.54]	0.045
G+: MLP meta + convexity	+0.24	[+0.02, +0.43]	0.015
F: CMDP-Lagrangian	+0.56	[+0.16, +0.98]	0.010
RL: Q-learning	+0.72	[-0.06, +1.52]	0.055
SPY	+0.45	[+0.06, +0.82]	0.015

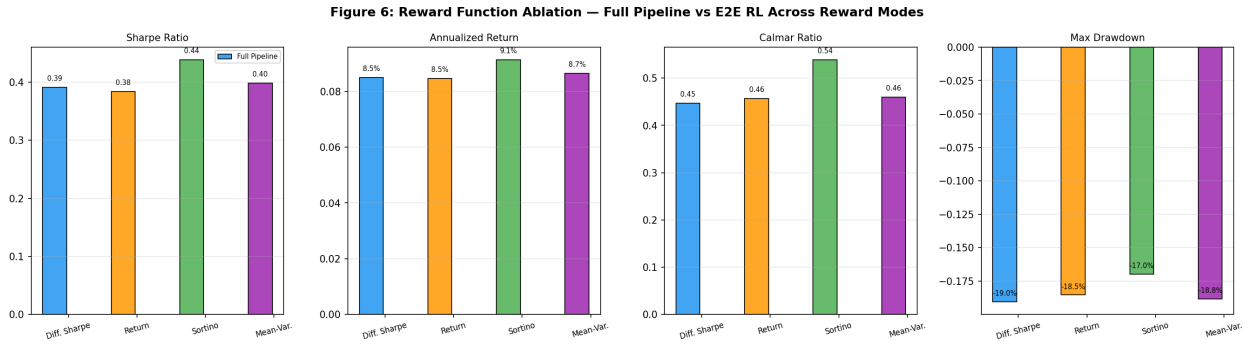

Figure 8: Supplementary reward-ablation figure for the revised bundle. Reward choice changes the controller's operating point modestly, but the broader ranking versus the factor-only benchmark and the strongest rule-based overlays is unchanged.

References

[1] R. Almgren and N. Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–39, 2000.

[2] H. Buehler, L. Gonon, J. Teichmann, and B. Wood. Deep hedging. *Quantitative Finance*, 19(8):1271–1291, 2019.

[3] T. Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327, 1986.

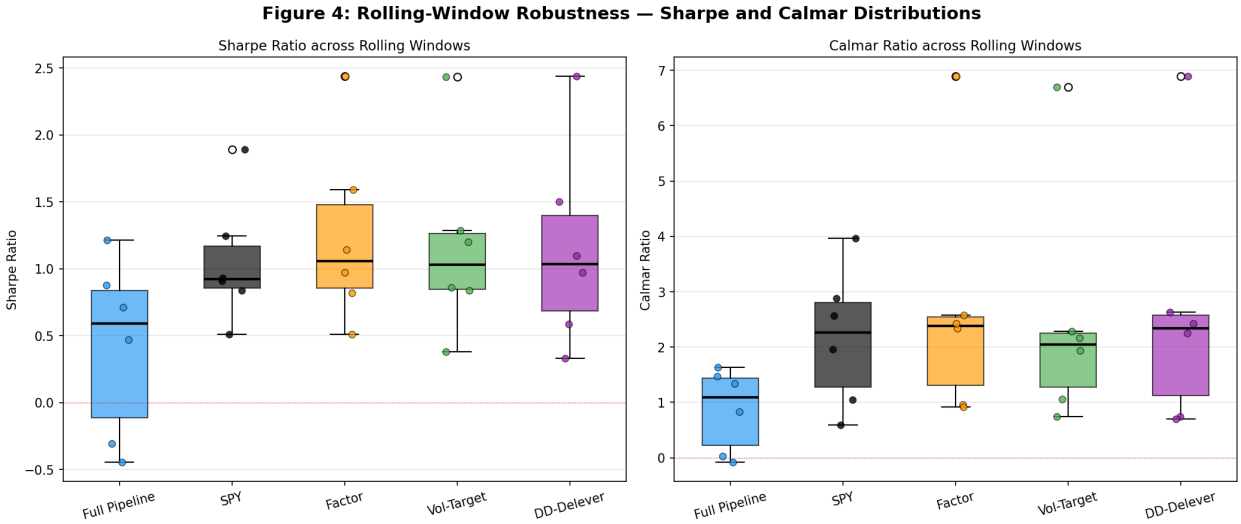

Figure 7: Appendix-only rolling-window robustness diagnostic for the retained legacy full pipeline. The pattern remains mixed: drawdown improves more often than Sharpe, and the windows should be interpreted as internal robustness slices from one historical sample rather than independent replications.

[4] L. Chen, M. Pelger, and J. Zhu. Deep learning in asset pricing. *SSRN Working Paper 3350138*, revised 2021.

[5] Y. Cong, B. Deng, X. Bao, and H. Dai. AlphaPortfolio: Direct construction through deep reinforcement learning and interpretable representation learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(6):4831–4839, 2021.

[6] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai. Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3):653–664, 2017.

[7] Y. Du, P. N. Kolm, A. Ritter, and T. Wang. Deep reinforcement learning for option replication and hedging. *The Journal of Financial Data Science*, 2(4):44–59, 2020.

[8] R. F. Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4):987–1007, 1982.

[9] E. F. Fama and K. R. French. Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56, 1993.

[10] S. Giglio, D. Xiu, and D. Zhang. Machine learning in asset pricing. *Annual Review of Financial Economics*, 14:337–368, 2022.

[11] S. Gu, B. Kelly, and D. Xiu. Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5):2223–2273, 2020.

[12] J. D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

[13] J. Jander, H. H. Li, M. Schmid, and T. Pohl. Reinforcement learning in finance: From current practice to standards and benchmarks. *SSRN Working Paper 5703882*, 2025.

[14] N. Jegadeesh and S. Titman. Returns to buying winners and selling losers: implications for stock market efficiency. *Journal of Finance*, 48(1):65–91, 1993.

[15] Z. Jiang, D. Xu, and J. Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017.

[16] P. N. Kolm and G. Ritter. Modern perspectives on reinforcement learning in finance. *SSRN Working Paper 3449401*, 2019.

[17] P. N. Kolm and G. Ritter. Dynamic replication and hedging: A reinforcement learning approach. *The Journal of Financial Data Science*, 1(4):159–171, 2019.

[18] J. Moody and M. Saffell. Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4):875–889, 2001.

[19] Y. Nevmyvaka, Y. Feng, and M. Kearns. Reinforcement learning for optimized trade execution. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 673–680, 2006.

[20] D. Pacheco Aznar. Portfolio management: A deep distributional reinforcement learning approach. *SSRN Working Paper 4522084*, 2023.

[21] D. Schmeidler. Subjective probability and expected utility without additivity. *Econometrica*, 57(3):571–587, 1989.

[22] S. S. Wang. A class of distortion operators for pricing financial and insurance risks. *Journal of Risk and Insurance*, 67(1):15–36, 2000.

[23] M. E. Yaari. The dual theory of choice under risk. *Econometrica*, 55(1):95–115, 1987.

[24] S. Sun, M. Qin, W. Zhang, H. Xia, C. Zong, J. Ying, Y. Xie, L. Zhao, X. Wang, and B. An. TradeMaster: A holistic quantitative trading platform empowered by reinforcement learning. In *Advances in Neural Information Processing Systems 36: Datasets and Benchmarks Track*, 2023.

[25] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, second edition, 2018.

[26] Z. Zhang, S. Zohren, and S. Roberts. Deep reinforcement learning for trading. *The Journal of Financial Data Science*, 2(2):25–40, 2020.